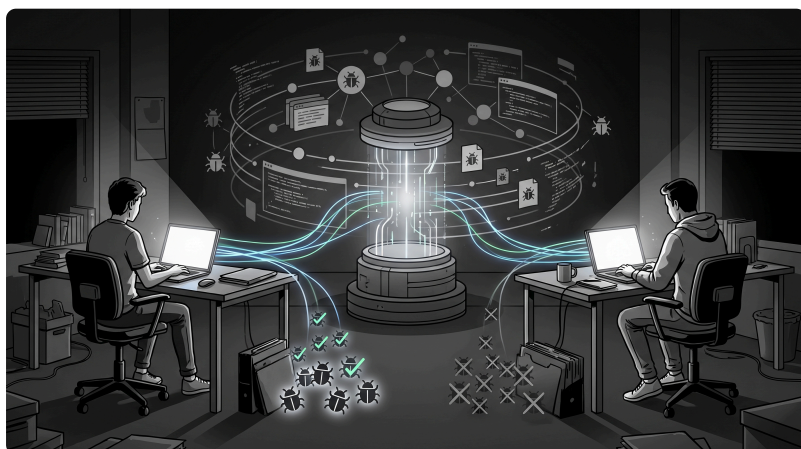




Joseph
Thacker

Сингулярность Bug Bounty: наш хакбот

01 июля 2026 года • взлом



В декабре этого года любой опытный хакер мог масштабировать своего хакерского агента, потратив сотни токенов, чтобы получить тысячи вознаграждений. Я называю это «сингулярностью Bug Bounty». Это история о том, как мы с Джей Ди ([xssdoctor](#)) создали хакбота, который за последние 5 месяцев нашел 126 уязвимостей.

Если вы хотите просто прочитать об ошибках, вы можете [сразу перейти к разделу об ошибках](#).

Оглавление

- История (автор JD)
 - Прототип
 - Правило хакбота № 1
 - Продолжайте взламывать!
 - Проверка
 - Оставайтесь в системе

- Реальное общение
 - БАГЗ
 - Обзор полученных результатов
 - Портфель исследований уязвимостей
 - Критическая уязвимость — полный захват партнерской платформы
 - Критическая уязвимость — утечка всех данных клиентов через API Western Union
 - Критическая уязвимость — полный захват учетной записи партнерской платформы через утечку OTP-хеша
 - Критическая уязвимость — XSS на raydium.io → примитив для опустошения кошелька каждого пользователя Solana
 - БОЛЬШИНСТВО ДРУГИХ ОШИБОК
 - Обнаружены агентом Autonomous Cyber «FUZZ E» — за одну ночь
-

The Story (написано Джей Ди)

Сообщение в Discord пришло в 6:10 утра:

rez0: Want to build a hackbot?

Для rez0 не было чем-то необычным написать мне в 6 утра. В те дни мы много общались в течение дня. Мы хорошо подходили друг другу. Мне нравилось глубоко погружаться в тему, изучать JavaScript и механику приложения. rez0 же предпочитал «охватывать все», находя нишевые или недооцененные уязвимости, мимо которых большинство людей пролистывает, и связывая воедино странное поведение и забытые конечные точки. Нас объединяла любовь к искусственному интеллекту.

Наши разговоры в основном сводились к навыкам Claude Code, которые мы написали, или к ошибкам, которые мы обнаружили с помощью наших агентов. Обычно я указывал своим агентам на минимизированный JavaScript. Мои навыки основывались на анализе исходного кода, путях на стороне клиента и флагах функций. Rez0 создал потрясающие навыки для фаззинга, перечисления поддоменов и тестирования на проникновение.

Claude 4.6 изменил все. До этого агенты использовались для ускорения рабочего процесса. Но внезапно они начали находить ошибки самостоятельно. Мы указывали агенту цель, загружали наши навыки, и появлялись настоящие баги. Вопрос был уже не в том, смогут ли агенты помочь нам взломать систему. Вопрос был в том, смогут ли они заменить целые этапы нашего рабочего процесса и насколько это будет экономически выгодно.

Самое важное решение, которое мы принимаем как багхантеры, — это то, как распределить свое время. Что лучше: «уйти в глубину» и потратить часы или дни на

изучение механики приложения или «пойти вширь», потратив недели или месяцы на написание кода и поддержку системы автоматизации? Каждая минута, потраченная на разведку, — это минута, которую мы не тратим на взлом.

Но в отличие от людей, агенты ИИ не устают и не скучают. Агентам ИИ не нужно спать или есть. Они не прокрастинируют, просматривая по пять часов видео про Minecraft на YouTube (редактор Джозеф, перестань меня подкалывать, Джей Ди). Идея Rez0 была проста: создать систему автоматизации, которая будет постоянно проводить масштабную разведку, а затем углубляться в изучение каждой цели.

Сообщение в Discord пришло в 6:11:

```
Xssdoctor: Lets build a freaking hackbot!
```

Прототип

День 1: 9:11

```
xssdoctor: This is going to be easy
```

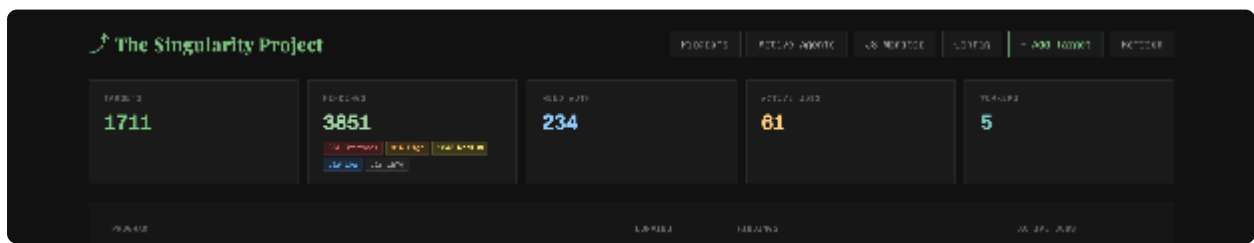
Это было непросто.

Поначалу эта идея казалась почти банальной. Весь прошлый год мы с вами совершенствовали навыки Claude Code и находили серьезные уязвимости. Навыки rez0 были сосредоточены на ошибках на стороне сервера и масштабной разведке. Мои навыки были направлены на глубокий анализ приложений и уязвимости на стороне клиента.

По отдельности эти навыки уже были полезны. Мы использовали их для ускорения собственных рабочих процессов. План состоял в том, чтобы объединить их в единую автономную систему: хакбота, способного одновременно проводить масштабную разведку и глубокий анализ.

Архитектура казалась довольно простой. Бот будет работать в облаке. Мы будем передавать ему список целей для программы Bug Bounty, и он будет последовательно работать с ними, загружая различные навыки в зависимости от того, что обнаружит. Навыки рекогносцировки позволят составить карту поверхности атаки. Навыки анализа позволят изучить логику приложения. Дополнительные агенты смогут проверять результаты, объединять ошибки в цепочки и автоматически составлять отчеты.

Конечно, поскольку мы не умели делать что-то наполовину, мы сразу же начали думать о дашбордах, красивых дашбордах, дашбордах, созданных с помощью Claude. Дашбордах, которые выглядели вот так.



Дашборд во всей своей красе, созданной с помощью Claude.

За три часа мы создали очереди, телеметрию, логи, систему отслеживания ошибок, оценку серьезности и оркестрацию агентов. Мы были готовы к работе.

Восемь часов спустя реальность дала о себе знать

Правило хакбота № 1

Мы проснулись и увидели на панели множество отчетов об ошибках, а четверть наших токенов исчезла. На несколько секунд мы поверили, что добились успеха. А потом начали читать отчеты.

Первая ошибка была связана с межсайтовым скриптингом. Вторая — с неправильной настройкой CORS, которую нельзя было использовать. Затем последовала «критическая уязвимость с возможностью удаленного выполнения кода», которая оказалась полной ерундой. Один за другим результаты проверки оказывались несостоятельными даже при минимальной проверке. Дашборд выглядел впечатляюще, но по сути представлял собой набор случайных данных.

Сначала мы были в замешательстве. Это были те же навыки, которые мы использовали на своих компьютерах. По отдельности они уже помогли нам найти реальные ошибки. Почему же они внезапно стали бесполезными, как только мы подключили их к облачной системе оркестрации? Ответ стал очевиден, когда мы увидели, как бот работает в режиме реального времени.

Когда мы взламывали системы вручную, мы постоянно управляли агентом. Мы замечали, когда он отвлекался на конечную точку с низким уровнем сигнала. Мы перенаправляли его на более интересный участок кода. Мы просили его повторить запрос, тщательнее изучить ответ или переключиться на совершенно другую поверхность атаки. Даже когда ИИ работал «автономно», мы все равно выступали в роли обратной связи. У хакбота ничего этого не было. Тогда мы и сформулировали первое правило для хакбота:

Правило Hackbot № 1: Всегда сохраняйте логи

Нам нужна была возможность отслеживать процесс принятия решений агентом. Не только итоговые отчеты, но и каждую выполненную команду, каждый отправленный запрос и каждый сделанный вывод.

Наше решение было простым: транслировать весь сеанс claude прямо в Discord

Теперь мы могли наблюдать за тем, как бот думает, в режиме реального времени, и логи стали более ценными, чем сами результаты. Оказалось, что разница между полезным хакерским инструментом и дорогостоящей машиной для генерации галлюцинаций заключается в наблюдаемости.

Продолжайте взламывать!

Первое, что мы поняли: бот почти ничего не взламывает.

Мы написали обширный файл [claudе.md](#), в котором подробно объяснялось, как должен вести себя бот, какие уязвимости атаковать в первую очередь и как использовать наши навыки. Теоретически у него было все необходимое. На практике он взламывал цель за двадцать минут и объявлял победу.

Поначалу его действия выглядели убедительно. Бот перечислял конечные точки, проводил разведку, тестировал несколько полезных нагрузок и генерировал подробный отчет. Но как только мы начали читать логи, стало очевидно, что происходит. Он останавливался на первом правдоподобном объяснении для всего. Если запрос не выполнялся, он двигался дальше. Если конечная точка казалась ему скучной, он ее игнорировал. Если баг почти срабатывал, он редко пытался использовать его по-другому. Люди так не взламывают. Когда мы работаем вручную, мы часами закидываемся на мельчайших деталях. Мы повторяем запросы с разными заголовками. Мы следуем странным редиректам. Мы вглядываемся в минимизированный код JavaScript, пока какой-нибудь скрытый флаг функции вдруг не обретает смысл. Большинство реальных ошибок возникает только потому, что человек не сдаётся после первого тупика. Это стало правилом № 2 для Hackbot

Правило хакбота № 2: Продолжайте взламывать

Решение заключалось не в том, чтобы сделать модель умнее, а в том, чтобы сделать ее более устойчивой. Существует множество способов продлить время работы агента, но мы остановились на методе, позаимствованном из рабочих процессов долгосрочного кодирования ИИ: циклах Ralph.

A ralph loop is deceptively simple. The agent performs a task, evaluates the result, generates new follow-up tasks and then repeats the cycle indefinitely until a stopping condition is reached. Instead of treating hacking like a single prompt-response interaction. The bot treats every finding as the beginning of another investigation.

This changed everything. Instead of spending twenty minutes on a target, the bot would spend hours digging deeper into anything even remotely suspicious. A weak signal was no longer ignored. It became a branch in the investigation tree. A reflected parameter became a search for hidden sinks. A strange redirect became a hunt for auth bypasses.

Most importantly, the bot started revisiting its own conclusions. One agent would decide an issue was not exploitable. Three loops later, another agent would come back with a

completely different perspective and prove that it was. The longer the loops ran, the more the system began to resemble an actual hacker mindset. Finally, the hackbot was becoming a hacker.

But after a while, we started to see the other side of persistence. A ralph loop doesn't know when to quit. On a rich target, hacking for hours was exactly what we wanted. On a thin target with almost no attack surface, that same persistence was a disaster. The bot would spend hours, and a pile of tokens, hunting for a bug that was never there.

So we moved away from pure ralph loops and put another bot between us and the worker: an orchestrator. Its job was to watch the worker hack and judge the target. If the attack surface looked weak, it cut the worker loose and moved on to the next target. If the attack surface looked good, it told the worker to keep hacking — and it would actively encourage it, with things like “there is definitely a bug here. find it!”

That one change fixed both failure modes at once. The bot stopped giving up early on good targets, and it stopped wasting tokens on bad ones. There was only one more problem that we needed to solve...

Validation

There is an almost endless steam of content related to the “death of bug bounty”. Most of it points to the same problem: **AI slop**.

After a few weeks running the hackbot continuously, we experienced what everyone else would soon experience:



The bot was finding interesting behavior. It was uncovering strange responses, unusual edge cases and occasionally even genuinely dangerous vulnerabilities. But it was also hallucinating constantly. A reflected parameter would become a fake XSS. A verbose error message would be an imaginary RCE. A harmless CORS configuration would somehow transform into “critical account takeover”.

At one point. The dashboard looked incredible. Thousands of findings. Security scoring. Auto-generated reports. But most of it wasn't real.

When we started measuring the output honestly, the numbers were brutal. Roughly 80% of the findings were false-positives.

That sounds catastrophic, but the strange thing was that the remaining 20% were real. The problem was not that the bot couldn't hack, but that it had no skepticism. Humans naturally validate their own ideas while hacking. We retry requests. We test assumptions from multiple angles. We ask ourselves whether something is actually exploitable or whether we just want it to be exploitable. Left alone, the agent did none of that. Once it convinced itself a bug existed, it would happily generate a beautiful report explaining why.

That's when rez0 had an idea. It was actually such a great idea, we made a rule about it

Hackbot rule number 3: Validate

We decided to make a validation bot. This bot had a singular purpose: aggressively disprove findings generated by the hacking agents. Instead of rewarding the model for finding vulnerabilities, we rewarded it for killing them.

If the hackbot claimed XSS, the validator attempted to break the exploit chain. If the bot reported SSRF, the validator looked for evidence the request never actually left the network boundary.

Most importantly, the validator had no emotional attachment to the original finding. It did not want to "please us" by finding a bug.

Our false positive rate dropped from 80% to about 60%.

Every meaningful finding needs an adversary. One bot hacked. Another bot validated. The more paranoid the system became, the more useful it became.

Stay Logged In

When we started, our hackbots were "going wide" but mostly finding duplicates and informationals. We knew what the problems were. We weren't authenticated. Any bug bounty hunter knows that the real money is post-auth.

The problem sounds trivial: just give the bot a session. In practice, almost nothing cooperates. Very few targets hand you a non-expiring cookie or auth token. The good ones rotate sessions aggressively, so a token you captured in the morning was worthless by lunch. And a bunch of programs actively fight anything that even smells like a bot logging in. Amazon in particular spends an enormous amount of engineering effort making sure automated logins simply do not happen.

The hardest part was that our bot lived on headless cloud boxes. And headless Chrome is exactly what every one of these anti-bot systems is tuned to detect and filter. The login page would load, the bot would type the right credentials, and the site would quietly decide it was a robot and drop a CAPTCHA or a “verify it’s you” wall in front of it. There are services like Browserbase that promise to beat the CAPTCHAs and pass as a real browser. We tried but could never get them to reliably work for us.

Then we looked at our metrics, and it was worse than we thought. At one point, **80% of our tokens were being spent on auth**. Not hacking. Not validating findings. Just trying, over and over, to log the bot into targets that did not want it logged in. We were burning the entire budget banging on front doors that slammed shut every time.

The fix, when we finally landed on it, was almost embarrassingly low-tech. Stop fighting the anti-bot systems on their terms. So we took one physical computer, a real machine with a real browser, and we built an agent that lived on it whose only job was to log in like a human — real Chrome, real profile, real fingerprint — and keep those sessions alive. When a token was about to expire, it quietly refreshed it. The cloud hackbot no longer touched login pages at all. It just asked the login agent for a live session and got to work.

The anti-bot systems were looking for headless Chrome on a datacenter IP. What they got was an ordinary browser on an ordinary computer, doing exactly what an ordinary user does. It turns out the winning move against a bot-detector is to not look like a bot.

Hackbot rule number 4: Let a real browser do the logging in

Once the bot could stay authenticated, the criticals started pouring in. Look at the bug list below — the IDORs, the broken access control, the write-primitives on other users’ objects. Almost none of that is reachable logged out. The single biggest jump in the quality of our findings did not come from a smarter model or a better skill. It came from the boring, stubborn work of keeping the bot logged in.

Real Talk

Clearly this wasn’t a breezy walk in the park, and we still have plenty of false positives. That said, we are **two guys**. And I’m a cardiologist full time. Since we began, it has gotten easier as well. Both codex and claude code added `/goal` mode, and opensource hacking agent systems (like strix) have been released over the last 6 months.

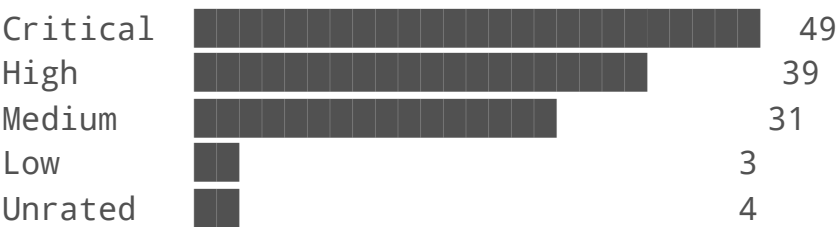
If we can do this, we believe that it massively increases the number of people who can use coding agents + harnesses to find critical vulnerabilities. And as you’ll see when you read the Google Platform takeover and the Western Union vulnerability below, there is tremendous real impact that can be had with these types of vulnerabilities.

THE BUGZ

An Overview of the Findings

Across the first half of 2026, the hackbot’s findings added up to **126 vulnerabilities** — spanning HackerOne programs, Google’s VRP, and direct disclosures (everything from the writeups below plus the 64 reports rez0 and I filed together on HackerOne). Bug bounty hunters love data, so here’s all of it, ugly stuff included.

Severity



88 of 126 findings (70%) were rated High or Critical.

How they landed

Outcome	Count
Accepted / resolved / triaged / in review	77
Duplicate	35
Informative / Not Applicable	14

Here’s the stat we’re proudest of: **112 of 126 (89%) were confirmed real** — either accepted by the program or closed as a duplicate. And a duplicate is *not* a miss. It means the bug was genuine and exploitable; someone just got there first. Only 14 were waved off as informative or N/A. For a system that two guys point at a target and walk away from, an 89% true-positive rate is amazing.

Our single largest bounty was **\$15,000** (a one-click account takeover). We still have one unpaid that could dwarf it though ;)

And a chunk of the High/Critical findings are still sitting in **triage**, waiting on the programs to pay.

Vulnerability Research Portfolio

Critical — Full Partner Platform Takeover

This is the coolest (and one of the most impactful) bugs I’ve ever found. And by “I”, I mean claude code, xssdoctor, and I. 😂

The full chain is actually really cool:

1. At some point in the past during all my Google api research (which you'll see below), this Google API key was slurped up into my Google API key database: `AIzaSyB...`
2. We tested it against the **Identity Toolkit** `getProjectConfig` endpoint, which leaked the Firebase project `partner-dri-prod` and its authorized domains — including `partnerdri.com` , `partner-companion.cloud.google` , and `delivery-readiness-portal.cloud.google`
3. Visiting `partnerdri.com` revealed the **Delivery Readiness Portal**, an internal Google partner management system
4. The Firebase project allowed **email+password signup without email verification**, so we registered an account.
5. An unauthenticated API endpoint (`GET /partner`) **leaked all 2,870 partner organizations and their 3,886 email domains** (already reported in report 495884998)
6. Of those 3,886 domains, **~170 had expired DNS registrations** and were available for purchase
7. We registered one of these expired domains, created a Firebase account with that domain email, and signed up on the DRI portal
8. When we logged in with google as that new admin, we had **partner** admin access
9. Once in there, as a normal user, we could see the DRI admin for that partner organization was so we set up a google workspace with **that** email.
10. We made another partner admin to do testing
11. Claude code was testing and figured out that one of the admins could POST `/dri/api/v1/user` with "role": "Google Super Admin" on the OTHER admin to escalate to a google super admin.
12. Login with that second admin you can now see EVERY user (like 60,000 and every org and manage/delete/approve/create)

Critical — Western Union API Leaks all Customers' Data

Yesterday, we discovered a critical vulnerability in Western Union's customer profile API. The endpoint at <https://www.westernunion.com/cusprofile/v2/cust/customers/multiparam> allows anyone to look up customer records by phone number with zero authentication.

A simple POST request with a phone number returns full customer PII including full name, email address, phone numbers, WU account number, and in some cases date of birth, home addresses, and government-issued document numbers. A single query can return multiple customer records.

Example request (no auth needed):

```
curl -sk \
  -H "Content-Type: application/json" \
  -H "x-wu-correlationId: test-001" \
```

```
-H "x-wu-externalRefId: test-001" \  
-X POST  
"https://www.westernunion.com/cusprofile/v2/cust/customers/mult:  
\  
-d '{"customer":{"phoneNumber":  
[{"countryCode":"1","number":"6068752453"}]}}'
```

I confirmed my own personal data is accessible through this endpoint. This allows enumeration of Western Union's entire customer database by iterating through phone numbers, exposing millions of customers' PII or very easily specifically targeting individuals.

Additionally, the same endpoint supports a name-based lookup that accepts a first name prefix, last name prefix, and date of birth. While the DOB parameter is required, its value is not validated against accounts that do not have a DOB stored, which appears to be a large portion of the customer base. This means an attacker can search by partial name with any arbitrary DOB and receive full PII for all matching accounts without a DOB on file.

Example:

```
curl -sk \  
-H "Content-Type: application/json" \  
-H "x-wu-correlationId: test-001" \  
-H "x-wu-externalRefId: test-001" \  
-X POST  
"https://www.westernunion.com/cusprofile/v2/cust/customers/mult:  
\  
-d '{"customer":{"name":  
{"first":"J","last":"T"},"dateOfBirth":"0000-00-00"}}'
```

This returns 49 customer records per request (the server-side cap), matching any last name starting with "T" regardless of first name or DOB value. By iterating through two-character last name prefixes (AA-ZZ), an attacker can systematically enumerate nearly all accounts without needing phone numbers.

We identified this endpoint through static analysis of the WU JavaScript bundle (<https://www.westernunion.com/staticassets/scripts/a4fe8b1434a298fb3e73c9735fc5406c.js>), which references a triggerCustomerLookup function pointing to this path.

Critical — Full Partner Platform Account Takeover via OTP Hash Leak

This one is wild. Same DRI portal as last time, but a totally different chain — this time we go from a leaked Google API key all the way to taking over **any existing user account** on the

portal by cracking a bcrypt-hashed OTP offline. And by “we”, I mean claude code, xssdoctor, and I. 😂

The full chain (same first few steps):

1. With the /partner leak, we identified likely real user emails (`firstname.lastname@partnerdomain.com` patterns) for any of the 2,870 partner orgs
2. We triggered a **password reset** for a target user via `POST /login/verifyWorkEmail`
3. We then called `POST /login/validateOtp` with a wrong OTP — the endpoint **leaked the bcrypt hash of the real OTP** in the error response
4. The OTP is an **8-digit number hashed with bcrypt cost 10** — crackable in ~2.5 hours on a T4 GPU or ~30 minutes on an A100
5. With the cracked OTP, we could complete the password reset and **take over the victim’s account**

Critical — Stored XSS on raydium.io → Wallet Drain Primitive on Every Solana User

This one is beautiful. It’s a stored XSS where the payload lives **on the Solana blockchain itself**, gets re-served by Raydium’s indexer forever, and ends in a one-click wallet drain against any user with auto-connect enabled.

The full chain:

1. While auditing the Raydium frontend bundle, we noticed a helper `ye(e)` in `chunks/9517.d81418fe83e1505c.js` that assigns user-controlled HTML to `innerHTML` on a **detached `<div>`** to “decode entities” — a classic “looks safe because React escapes the return value” anti-pattern
2. The catch: the browser still **parses the HTML and fires `onerror` synchronously** on detached nodes, so the return value of `ye()` is irrelevant — the XSS has already fired by the time the function returns
3. We traced the input and found the Comments component seeds a synthetic “creator’s first comment” using the launchpad token’s `description` field — `ye(t.text)` is called on that string verbatim
4. The `description` is pulled from `launch-mint-v1.raydium.io/get/by/mints`, which serves whatever the **Metaplex on-chain uri JSON** says — meaning the attacker controls it by minting a token
5. We minted a Solana token via the **Raydium Launchpad program** with `description: "<img src=x onerror=...>"` in its IPFS metadata — for a few cents in fees we now have a persistent stored XSS payload **on-chain, immutable, served by Raydium’s own indexer**
6. Opening `https://raydium.io/launchpad/token/?mint=<OUR_MINT>` in any browser executes attacker JavaScript in the `raydium.io` origin — no CSP, no Trusted Types, no sanitization at any layer

7. From in-origin, the payload `hooks window.solana.signTransaction / signAndSendTransaction / signAllTransactions` — Phantom, Backpack, Solflare, and the official Solana wallet adapter all behave identically here
8. Every subsequent transaction the victim authorises through Raydium (swap, LP, claim) is silently substituted with a `SystemProgram.transfer` that drains their **full SOL balance** to the attacker
9. The user *does* click “approve” in their wallet — but for a transaction that doesn’t match their UI intent. This is exactly the primitive that powers production Solana drainer-as-a-service campaigns
10. Mass distribution is trivial — a tweet of `https://raydium.io/launchpad/token/?mint=<MINT>` is **indistinguishable from how every legitimate Raydium Launchpad token is shared today**. URL allowlists, EDR, and corporate proxies all whitelist `raydium.io`

The persistence story is the wildest part: the malicious `description` lives in Metaplex Token Metadata, which is **immutable** for tokens minted via the Raydium Launchpad program. Raydium can’t edit it out at the source — only block-list the mint at the indexer or patch the frontend. The on-chain payload outlives every fix that doesn’t ship to the frontend.

Two live PoC mints on Solana mainnet, both indexed and reachable:

- **Alert PoC:** `https://raydium.io/launchpad/token/?mint=Gjm1kBiCbfx8LfCAgswHFdFzKs7uogCNUw7yCjVaocfY` — pops `alert(document.domain)` on visit
- **Wallet-drain PoC:** `https://raydium.io/launchpad/token/?mint=3XkkorkMhMZDJtpcwd3ESC8WBRKxH3xHXukDUA4MoKWU` — imports the escalation payload from `poc.xssdoctor.com`, arms the wallet hook, drains on the next Raydium signing prompt

MOST OF THE OTHER BUGS

IAJQP018	Critical — Unauthenticated API on a major GPU and AI chip manufacturer’s autonomous vehicle simulation platform exposed employee PII, internal roles, and partner relationships. Escalated from information disclosure to full admin access — the same unauthenticated endpoints allowed creating, updating, and deleting users, enabling self-promotion to admin and full platform takeover.
#2fd5d29c	Critical (P1, Duplicate) — Full OAuth account takeover of a major enterprise productivity and collaboration suite’s AI MCP server. Public-client Dynamic Client Registration accepted with no <code>redirect_uris</code> validation; the platform’s own <code>/v1/authorize</code> page rendered the attacker-controlled <code>client_name</code> and policy/TOS URIs as fully-branded consent text with no third-party-application warning; the UI showed three product checkboxes while the server actually requested 12 OAuth scopes from the IdP (including write to the issue-tracking and wiki products plus <code>offline_access</code>); and <code>state</code> was unsigned base64 JSON containing the attacker’s <code>redirect_uri</code>. After IdP authentication the callback 302’d the authorization code straight to the attacker, who exchanged it at <code>/v1/token</code> with no client secret (public

		client) and received a bearer + refresh_token granting full read/write to the victim's tenants via the MCP JSON-RPC endpoint.
#de29505b		Low (P4, Triaged) — Authorization bypass on a major US home and commercial security monitoring provider's incident API: a body-supplied <code>clientId</code> was passed straight through to an Oracle stored procedure as the acting employee number, without ever being bound to the <code>cid</code> claim in the gateway-validated OAuth token. A single read-scoped client token could forge any employee number on writes to incidents, comments, issues, and employee assignments — attributable to any employee, attached to any real customer or monitored site — while a differential error oracle on the same field enumerated the entire valid employee/customer/site ID space.
#6d456454		Critical (P1, New) — Read+write IDOR chain on the same security monitoring provider's customer search API. <code>POST /cxo-exp-api/v1/api/customers/search</code> accepted arbitrary numeric customer IDs and returned full customer, site, and security-system records (phone numbers, site IDs, security-system IDs, central-station IDs, system/equipment types, service status, panel location, maintenance flags). The disclosed <code>siteId</code> was then accepted by <code>PUT /cxo-exp-api/v1/api/sites/{siteId}/details</code> under the same read-scoped engagement token, allowing modification of monitored-site detail fields on unrelated customer accounts (validated by changing <code>crossStreet</code> to a marker and restoring the original value).
#4a54b3a2		Critical (P1, New) — Read+write IDOR on the same provider's site-contacts API: <code>GET /cxo-exp-api/v1/api/contacts/sites/{siteId}</code> returned emergency/dispatch contact records and cleartext 4-digit <code>personalIdentificationCode</code> (PIC) values for arbitrary numeric site IDs, and <code>POST /cxo-exp-api/v1/api/contacts/sites/{siteId}</code> accepted <code>PIC-UPDATE</code> operations under the same read-scoped token — letting an attacker rewrite the PIN used to authenticate alarm-cancellation calls on any monitored site (validated by mutating and restoring a target PIC).
#8af0cf1b		Medium (P3, Triaged) — IDOR on the same provider's IVR billing API: <code>/ivr-exp-api/v1/api/billing/{customerId}</code> had no object-level authorization on the <code>customerId</code> path parameter, so a read-scoped engagement bearer token retrieved <code>billingDetail</code> , <code>recentPaymentDetail</code> , and <code>easypayAccounts</code> recurring-payment-method metadata for arbitrary numeric customer IDs.
#83b71966	Critical (P1)	\$5,500 — Unauthenticated mass PII and internal sales-data disclosure on a major US cable, broadband, and advertising conglomerate's internal advertising-sales CRM. The production SPA host shipped a committed <code>/dummyData/</code> directory containing five <code>.json</code> files (915 order records totaling ~\$5.3M of contracted business) — discovered by pulling live production JS sourcemaps and grepping <code>sourcesContent</code> for the commented-out mock paths developers had left behind in <code>orderService.js</code> / <code>adCopyService.js</code> . The files exposed real employee names (cross-validated against LinkedIn), internal SAM-style usernames, advertiser and agency relationships, contracted-dollar amounts, flight dates, and internal ad-copy preview URLs — with identical exposure on the <code>dev</code> , <code>qa</code> , and <code>stg</code> mirrors of the same host.

#89805a41		Low (P4, Duplicate) – Zero-click OAuth authorization code disclosure on a major image-sharing social platform’s enterprise SSO and LINE OAuth callback pages. <code>/sso/callback</code> and <code>/oauth/line/redirect</code> fired <code>window.opener.postMessage(querystring, "*")</code> with the full query string (including <code>code</code> and <code>state</code>) to any cross-origin opener, while the platform served <code>Cross-Origin-Opener-Policy</code> in report-only mode site-wide so <code>window.opener</code> remained intact across origins. An attacker page opening the IdP authorize URL (with the platform’s server-pinned <code>redirect_uri</code>) as a popup received the victim’s authorization code directly, completing account takeover via the platform’s normal SSO login flow.
#ec148e0c	Critical (P1)	\$5,000 – Full unauthenticated CRUD on a major US consumer financial services company’s third-party point-of-sale lending partner’s production payment API. The <code>/localCharge</code> and <code>/dashboard</code> services on the Heroku-hosted backend were missing authentication entirely – any request with <code>Content-Type: application/json</code> could read, modify, delete, or create entries in the 4.5-million-record production charge database, including the most recent live charges. The test environment was identically vulnerable across <code>/api/v1/platform/onboarding</code> , <code>/account</code> , <code>/duplicate-charges</code> , and <code>/balance</code> (full Stripe Balance disclosure).
#b6beea7d	Critical (P1)	\$5,000 – Mass unauthenticated PII disclosure on a major US consumer financial services company’s retailer-app admin backoffice. <code>GET /api/<release_id>/<CTID>/feedback</code> was the single route in the admin API tree missing authentication, while every sibling endpoint enforced Okta SAML SSO. Enumerating the sequential <code>CTID</code> path segment dumped 492K+ customer feedback records spanning dozens of national retail and payment brands – including 122K emails, 539 addresses, 30 DOBs, 8 full SSNs, 342 credit card numbers, 4 passwords, and 6 routing/bank account numbers pasted into the feedback bodies themselves.
#32d0ccf5		Critical (P1, Out of Scope) – Three chained vulnerabilities on a major US quick-service restaurant chain’s employee casting-call portal (partner-run, technically out of scope, disclosed in good faith): (a) unauthenticated <code>/admin/api/fs/cr</code> returned a valid Filestack policy + signature granting full read/write to the brand’s S3 bucket holding 126+ employee video submissions; (b) a hidden <code>/isolated/register</code> route (discovered via X-Forwarded-Host header injection leaking the Ziggy route map) auto-granted view-only admin to the submissions dashboard, exposing PII for all 191 employee applicants; (c) unauthenticated IDOR on <code>/admin/download/image/{id}</code> served sequential employee headshots with the employee’s full name in the <code>Content-Disposition</code> filename across IDs ranging from 3 to 3560+.
#733b481c		High – Cross-origin postMessage XSS in a major US quick-service restaurant chain’s AI chat widget: the <code>updateConfig</code> handler processed messages with no origin check and accepted a <code>ui.customJS</code> field that was injected as a <code><script></code> element inheriting the CSP nonce from <code>html[data-nonce]</code> . With no <code>X-Frame-Options</code> or <code>frame-ancestors</code> directive, the page was embeddable in any attacker iframe – yielding zero-click arbitrary JS execution on the chat-widget subdomain and, via cookie tossing onto the parent registrable domain, session fixation and login CSRF reachable from every brand subdomain (ordering, services, marketing, etc.).

#498679301	High (P2/S2)	\$7,500 — Server-side OAuth scope escalation on a major search engine's Gmail MCP server: bearer tokens scoped to only <code>gmail.readonly</code> + <code>gmail.compose</code> could TRASH, SPAM, or remove INBOX from arbitrary threads via the MCP <code>label_thread</code> tool, while the raw Gmail API correctly enforced <code>gmail.modify</code> and returned 403 for the same token. Reward includes a 50% bugSWAT bonus on the \$5,000 base.
#497948709	High (P1/S1)	\$500 (split) — Unauthenticated mass partner-contact PII extraction on a major search engine's small-business education community (Salesforce Experience Cloud). A custom Apex controller exposed to the guest profile returned <code>Contact__r.Name</code> and the full Account relationship without <code>with sharing</code> , so chaining unauthenticated GraphQL enumeration of ~26,773 Event records against the Apex resolver yielded full legal names, organizations, venue addresses, and multi-year per-person activity histories for thousands of partner contacts.
#497941127	Medium (P1/S1)	\$240 — Reflected XSS on a major search engine's cultural archive web app: the 3D Pottery experiment base64-decoded a <code>cp</code> URL query parameter as JSON and assigned the <code>score</code> field directly to <code>#sharescore.innerHTML</code> . Reward reduced because the sink landed inside a sandboxed acquisition-tier embed iframe.
#491972242	Critical (P2/S2) — Broken tenant isolation in a global procurement network exposed 135 million records — including a major technology company's procurement messages, employee directories (498 employees with names and corporate emails), supplier relationships, and full conversation content — to any authenticated user with a free account. Demonstrated targeted extraction of the technology company's procurement staff, job titles, direct phone numbers, and purchase order discussions.	
#3712304	Critical — Unauthenticated <code>GET /sdk/{envId}/settings</code> on a major Web3 wallet authentication provider exposed customer-private RPC API keys (Infura, Alchemy, and other paid-tier credentials) for 141 customer environments. The same response leaked webhook configuration and signing material; webhook creation and transaction-relay paths were confirmed end-to-end.	
#3707975	Critical — Unauthenticated <code>POST /admin-api/migrate</code> on a major enterprise secure messaging platform's admin console accepted integer <code>nonce</code> / <code>session</code> pairs and returned the victim's full account record along with a freshly-minted, server-valid session cookie. The issued cookie authenticated as the victim against <code>/admin-api/*</code> , granting full administrative takeover of the victim's enterprise messaging networks.	
#3691764	High (Duplicate) — Improper authorization on <code>PATCH /member-collaborators/<id></code> on a major US financial institution's lifestyle services platform allowed an inviter to silently change a consented delegate's email address post-acceptance. The Android UI visually disabled the email field, but the backend did not enforce that restriction, redirecting all future reservation and concierge correspondence to an address the original invitee never agreed to.	
#3691653	High (Duplicate) — Unauthenticated <code>PATCH /member-delegates/<UUID>/consent</code> on a major US financial institution's lifestyle services consumer gateway treated possession of	

	the <code>collaboratorId</code> (visible to the inviter before the invitee ever clicked the email) as proof of consent, letting the inviter flip <code>consentStatus: Pending → Accepted</code> server-side on behalf of any invited email and begin receiving reservation copies addressed to that delegate.
#3691582	Medium (Duplicate) — Mass assignment on <code>PATCH /v2/members/me/profile</code> on a major US financial institution's lifestyle services member gateway accepted arbitrary top-level keys without an allowlist, letting unprivileged members overwrite restricted fields including <code>email</code> , <code>businessId</code> (tenant assignment), <code>referralCode</code> , <code>referrerMemberId</code> , and <code>onboardingStatus</code> .
#3689404	Medium (Duplicate) — Cross-org IDOR on <code>/org/{ORG}/metrics</code> on a major US financial institution's Drupal-based developer portal: every other org-scoped route enforced membership, but the metrics route did not. Any authenticated portal user could fetch any organization's display name, total hit counts, error rates, latency averages, and full application inventory.
#3688351	Medium (Duplicate) — Cross-merchant IDOR on <code>/api/merchant/settlements</code> and <code>/api/merchant/settlement-details</code> on a major US financial institution's merchant smartview portal: supplying any <code>merchantNo</code> returned full transaction-level records — customer names, invoice numbers, PO numbers, fee amounts, net amounts — for every merchant on the platform, with no authorization check tying the user to the requested merchant.
#3688246	Medium — Cross-merchant IDOR on <code>/api/merchant/applications</code> on a major US financial institution's merchant smartview portal returned all 507 credit applications across every merchant from a single authenticated session, leaking applicant PII (company name, contact name, full address, phone, fax, email) plus credit-decision data (approval status, decision date, approved credit limit, internal application ID).
#3671934	Medium — Contentful CMS Preview API token (alongside the Delivery token) exposed in publicly accessible JavaScript source maps on a major prescription pricing and telehealth platform's production CDN. The Preview token granted read access to 271K+ CMS entries including 89K unpublished drafts containing internal pharmacy routing, copay card configuration, and business workflow data.
#3648085	Critical (Duplicate) — Anonymous Firebase authentication enabled on an on-demand commercial insurance platform's feature-flags Firebase project, combined with overly permissive Firestore rules, granted unauthenticated read access to the internal <code>users</code> collection — exposing 24 employee records with full names, corporate and personal emails, Google profile photo URLs, and internal RBAC permission mappings.
#3621470	High — Unauthenticated Sitefinity CMS OData endpoint <code>/api/default/socialmediachannels</code> on a global food and beverage conglomerate's Brazilian site exposed live non-expiring Facebook Graph API Page Access Tokens with <code>pages_manage_posts</code> scope for multiple official verified corporate pages (over 1M combined followers across regional accounts), enabling unauthorized publish/edit/delete on the brand's social presence.

#3616733	Critical — Authentication bypass vulnerability in a major cryptocurrency wallet provider's third-party authentication API. An IDOR allowed unauthorized access to any user's wallet account by manipulating object references.	
#3600887	High — Unsafe deserialization in a widely-used Java ORM library (maintained by an open-source security company) enabling remote code execution via the serialization helper class.	
#3597005	High — IDOR on a major cloud and AI provider's internal conversation management service, allowing unauthorized access to conversation data across accounts.	
#3596433	Critical — Write IDOR on a major credit card company's partner portal allowing any authenticated user to take over any project by manipulating project references in API requests.	
#3596129	High — Null byte injection on a major credit card company's partner portal bypassed access controls to dump all projects and their metadata from the platform.	
#3596095	Critical — Null byte IDOR on a major credit card company's identity platform dumped every user in the organization's Okta tenant, exposing the full employee and partner directory.	
#3596077	High — Self-registration bypass on a major credit card company's invite-only partner portal via direct interaction with the Okta IDX API, circumventing the intended enrollment flow.	
#3595091	High — Open OAuth Dynamic Client Registration on a major credit card company's authorization server, allowing anyone to register arbitrary OAuth clients without authentication.	
#3576062	Critical	\$6,561 — Hardcoded bearer tokens discovered in client-side JavaScript bundles of an IT management and cybersecurity platform, enabling unauthenticated enumeration of all customer devices via the heartbeat API.
#3558966	Critical — IDOR on a major e-commerce and cloud provider allowing modification of another user's resources via direct object reference manipulation.	
#3558957	High — Missing authentication on internal domains of a major e-commerce and cloud provider's AI conversation platform leaked internal employee conversations and support data.	
#3555405	Critical — SSRF and local file read on a global food and beverage conglomerate's regional marketing site, enabling server-side requests to internal resources and reading arbitrary files from the server.	
#3555227	Critical — Firebase anonymous authentication on a global food and beverage conglomerate's rewards platform granted full Firestore CRUD access, allowing any unauthenticated user to read, write, and delete all reward program data.	

#3555180	Medium	\$500 — Wide-open Drupal JSON:API on a global food and beverage conglomerate's regional brand site exposed user emails, internal file paths, and the complete content architecture.
#3548491	Medium	Broken access control on a major US financial institution's platform where a secondary admin could demote or modify other administrators including the business owner via direct API calls.
#3540573	High	\$4,525 — SSRF combined with information disclosure in an agricultural and industrial equipment manufacturer's directory application, enabling server-side requests to internal infrastructure.
#3535592	Medium	Production InfluxDB instance at an agricultural and industrial equipment manufacturer with debug endpoints publicly exposed, leaking internal metrics, database schema, and system configuration.
#3533940	Medium	\$2,500 — A major short-form video platform's developer portal was leaking other users' search queries (including explicit/NSFW content) through a misconfigured search endpoint.
#3522540	Medium	Unauthenticated GraphQL mutations on an agricultural and industrial equipment manufacturer's platform allowed snapshot deletion, with full schema and architecture disclosure via introspection and verbose stack traces.
#3510176	Medium	\$500 — Data exfiltration via markdown image rendering on a supply chain management software platform, where crafted markdown content triggered outbound requests leaking sensitive data to attacker-controlled servers.

Found by Autonomous Cyber's "FUZZ E" agent — single overnight run

This wasn't our hackbot, but I wanted to surface it in this blog post. The following findings were surfaced by [Autonomous Cyber's](#) autonomous offensive agent "FUZZ E" in one overnight run against targets they let me point it at. Absolutely insane findings on super hardened targets. You should 1000% check out their work.

#483668814	High (P2/S2)	\$5,000 — XSS in a major open-source web framework's internationalization system: the ICU message parser skipped URL sanitization for static HTML attributes in translation files, allowing a malicious translator to inject <code>javascript:</code> URLs into <code><a href></code> tags inside ICU plural/select cases and bypass the framework's documented sanitizer for the affected locale. Patch submitted and merged by the framework team.
#3550286	High	\$4,580 — Arbitrary file read in a major enterprise software company's open-source e-commerce platform, allowing attackers to read sensitive server-side files via remote file inclusion.
#483679874	High (Duplicate)	SSRF in the same open-source web framework's server-side rendering module: the default HTTP interceptor blindly trusted the <code>Host</code> header to rewrite all

	relative URLs, enabling attacker-controlled outbound requests from the SSR process to internal networks, cloud metadata endpoints, and arbitrary external domains.
--	--

[Sign up for my email list](#) to know when I post more content like this. I also [post my thoughts on Twitter/X](#).

■ Ai ■ Hacking ■ Cybersecurity

Related Posts

it's important to say things 22 Jul 2026

Launching: rez0's rascals 17 Jul 2026

Operation Floodlight 01 Jul 2026